

Cryptic Chronicles: History of code and Ciphers

Submitted to the

Department of Master of Computer Applications in partial fulfilment of the
requirements for the Mini Project in

Web Programming – MCA26

by

Adithya Sree I V

1MS24MC002

Under the Guidance of

Mr. Sreehari T M

RAMAIAH INSTITUTE OF TECHNOLOGY

(Autonomous Institute, Affiliated to VTU)

Accredited by National Board of Accreditation & NAAC with 'A+' Grade

MSR Nagar, MSRIT Post, Bangalore-560054

www.msrit.edu

2025

**DEPARTMENT OF MASTER OF COMPUTER
APPLICATIONS**

CERTIFICATE

This is to certify that the project entitled “**Cryptic Chronicles: History of code and Ciphers**” is carried out by **Adithya Sree I V- 1MS24MC002** student of 2nd semester, in partial fulfillment for the Mini Project in Full Stack Development- MCA26, during the academic year 2024-2025.

MR. Sreehari T M
Guide

Dr. S Ajitha
Head of the Department

Name of Examiners

- 1.
- 2.

Signature with Date

ACKNOWLEDGEMENT

With deep sense of gratitude, I am thankful to our Respected HoD, **Dr. S Ajitha** for her pillared support and encouragement. I would like to convey my heartfelt thanks to my Project Guide **Mr. Sreehari T M**, Department of Master of Computer Applications, for giving me an opportunity to embark upon this topic and for her/his constant encouragement.

It gives me great pleasure to acknowledge the contribution of all people who helped me directly or indirectly realize it. First I would like to acknowledge the love and tremendous sacrifices that my Parents made to ensure that I always get an excellent education. For this and much more, I am forever in their debt.

I am thankful to all my friends for their moral and material support throughout the course of my project. Finally, I would like to thank all the Teaching and Non-Teaching Staff of Department of Master of Computer Applications for their assistance during various stages of my project work.

Adithya Sree I V

ABSTRACT

Cryptic Chronicles: History of Code and Ciphers is a full-stack web application that highlights the evolution of cryptography, from ancient techniques to modern encryption. Built with **Node.js**, **Express**, and **MongoDB**, the backend ensures efficient data handling, while **Tailwind CSS** powers a responsive and engaging frontend.

Designed to be both educational and interactive, the site presents a curated journey through the history of ciphers, combining technical development with rich historical content. It offers users a deeper understanding of how cryptography has shaped secure communication across eras.

.

Table of Contents

1. Introduction	1
1.1. Problem Definition	1
2. Implementation	2
2.1. Code snippets	2
3. User Interface Screenshots	10
4. Conclusion and Future Enhancement	13

1. Introduction

Cryptography has played a crucial role in communication security throughout history, evolving from simple substitution ciphers to complex encryption algorithms used in modern cybersecurity. Understanding the history and principles of cryptographic techniques not only provides insight into their impact on society but also helps in appreciating their relevance in today's digital world.

Cryptic Chronicles: History of Code and Ciphers is a full-stack web application designed to educate users on the evolution of cryptography while offering an interactive experience through a built-in cipher decoding mini-game. The site doubles as both an informational resource and a hands-on tool, making the world of codes engaging and accessible.

1.1 Problem Definition

Despite the widespread use of cryptography in modern technology, many people lack an understanding of its historical development and fundamental principles. Traditional learning methods often focus on theoretical aspects without providing practical engagement, making it difficult for users to grasp the significance and mechanics of encryption.

The challenge lies in presenting cryptographic history in an interactive and engaging manner that appeals to both casual learners and enthusiasts. *Cryptic Chronicles* addresses this issue by combining educational content with a mini-game that encourages users to decode ciphers, reinforcing their learning through hands-on experience. The website aims to bridge the gap between historical knowledge and practical application, making cryptography more accessible and enjoyable.

Furthermore, while there are numerous resources available on cryptography, many are either too technical for beginners or lack interactive elements to maintain engagement. A well-structured, visually appealing, and interactive platform can enhance learning by allowing users to explore cryptographic techniques at their own pace.

2. Implementation

The implementation of **Cryptic Chronicles: History of Code and Ciphers** involved developing a full-stack website using **Node.js**, **Express**, **MongoDB**, and **Tailwind CSS** to provide an engaging, interactive experience. Key sections include a rich gallery showcasing visual content, a cryptography timeline, and a cipher-based mini-game where users can **encrypt and decrypt messages** using various techniques.

HTML and **Tailwind CSS** ensure structured content and responsive design, while **JavaScript** enables dynamic interactions, particularly within the game module. The backend, built with **Node.js** and **MongoDB**, manages data storage and logic for game functionalities and user interactions. Emphasizing a mobile-first approach, the site features smooth navigation, intuitive layout, and clear instructions that foster an enjoyable learning environment

2.1 Code snippets

```
// src/pages/Cipher.jsx

import React from 'react';

import CipherTool from '../components/CipherTool';

export default function Cipher() {

  return (

    <div className="bg-[#fdf3e7]/90 rounded-xl shadow-xl border border-yellow-800 p-8
max-w-4xl mx-auto">

      <h2 className="text-2xl font-bold mb-4">Caesar Cipher Tool</h2>

      <CipherTool />

    </div> ); }

//Game.jsx

import { useState } from 'react';

import { motion } from 'framer-motion';
```

```
export default function Game() {

  const [word, setWord] = useState("");

  const [shift, setShift] = useState(1);

  const [encrypted, setEncrypted] = useState("");

  const [guessed, setGuessed] = useState("");

  const [showGuess, setShowGuess] = useState(false);

  const [showResult, setShowResult] = useState(false);

  const caesarEncrypt = (str, shift) => {

    return str

      .split("")

      .map((char) => {

        if (/[a-z]/.test(char)) {

          return String.fromCharCode(((char.charCodeAt(0) - 97 + shift) % 26) + 97);

        } else if (/[A-Z]/.test(char)) {

          return String.fromCharCode(((char.charCodeAt(0) - 65 + shift) % 26) + 65);

        } return char; })

      .join("");

  };

  const handleEncrypt = () => {

    const result = caesarEncrypt(word, parseInt(shift));

    setEncrypted(result);

    setShowGuess(true);

    setShowResult(false);

  };

  const handleGuessSubmit = () => {

    setShowResult(true);
```

```

    };

    const resetGame = () => {

        setWord("");

        setShift(1);

        setEncrypted("");

        setGuessed("");

        setShowGuess(false);

        setShowResult(false);

    };

    return (

        <div className="flex justify-center items-center min-h-screen w-full px-4 bg-fixed bg-
cover bg-center" style={{ backgroundImage: 'url(/assets/s4.webp)' }}>

            <motion.div

                initial={{ opacity: 0, y: -20 }}

                animate={{ opacity: 1, y: 0 }}

                transition={{ duration: 0.8, ease: 'easeOut' }}

                className="bg-yellow-100/10 backdrop-blur-md border-2 border-yellow-700 rounded-
xl shadow-2xl p-8 max-w-2xl w-full text-center font-serif"

            >

                <h2 className="text-3xl md:text-4xl font-bold text-yellow-900 mb-6 tracking-tight
border-b pb-2 border-yellow-700 font-cinzel drop-shadow-[1px_1px_0_rgba(0,0,0,0.7)]">

                    Caesar Cipher Challenge

                </h2>

                <input

                    type="text"

                    value={word}

                    onChange={(e) => setWord(e.target.value)}

```

```

        placeholder="Enter a secret word..."

        className="w-full p-3 mb-4 rounded-md border border-yellow-700 text-yellow-900
font-medium bg-[#fff8dc]/80 shadow-inner focus:outline-none focus:ring-2 focus:ring-
yellow-500 transition"

    />

    <div className="mb-4">

        <label className="text-yellow-900 font-medium mr-2">Shift:</label>

        <input

            type="number"

            value={shift}

            onChange={(e) => setShift(e.target.value)}

            className="w-16 p-2 rounded-md border border-yellow-700 bg-[#fff8dc]/80 text-
yellow-900 text-center focus:outline-none focus:ring-2 focus:ring-yellow-500 transition"    />

    </div>

    <motion.button

        whileHover={{ scale: 1.05 }}

        whileTap={{ scale: 0.95 }}

        onClick={handleEncrypt}

        className="mt-2 bg-yellow-700 hover:bg-yellow-800 text-white py-2 px-6 rounded-lg
shadow-md transition duration-300 font-semibold">

        Encrypt

    </motion.button>

    {encrypted && (

        <motion.div

            initial={{ scale: 0.8, opacity: 0 }}

            animate={{ scale: 1, opacity: 1 }}

            transition={{ type: 'spring', stiffness: 200, damping: 15 }}

```

```

        className="mt-6 p-4 bg-yellow-100/80 border border-yellow-700 rounded-md text-
yellow-900" >

        Encrypted word: <strong>{encrypted}</strong>

    </motion.div>

)}

{showGuess && (

    <div className="mt-6">

        <input

            type="text"

            value={guessed}

            onChange={(e) => setGuessed(e.target.value)}

            placeholder="Guess the encrypted word"

            className="w-full p-3 rounded-md border border-yellow-700 text-yellow-900 font-
medium bg-[#fff8dc]/80 shadow-inner mb-2 focus:outline-none focus:ring-2 focus:ring-
yellow-500 transition" />

        <motion.button

            whileHover={{ scale: 1.05 }}

            whileTap={{ scale: 0.95 }}

            onClick={handleGuessSubmit}

            className="mt-2 bg-yellow-600 hover:bg-yellow-700 text-white py-2 px-6
rounded-lg shadow-md transition duration-300 font-semibold" >

            Submit Guess

        </motion.button>

        {showResult && (

            <p className={`mt-3 font-bold ${guessed === encrypted ? 'text-green-700' : 'text-
red-700'}`}>

                {guessed === encrypted ? '🎉 Correct!' : '❌ Try again'}</p> )} </div> )}

```

```
<button onClick={resetGame}
  className="mt-6 bg-red-700 hover:bg-red-800 text-white py-1 px-4 rounded-md
transition>

  Restart

</button>

</motion.div>

</div>);}
```

Contact.jsx

```
import React, { useState, useEffect, useRef } from 'react';

export default function Contact() {

  const [focusedInputPos, setFocusedInputPos] = useState(null);

  const quillRef = useRef(null);

  useEffect(() => {

    if (focusedInputPos && quillRef.current) {

      const { x, y } = focusedInputPos;

      quillRef.current.style.animation = 'none';

      quillRef.current.style.transition = 'transform 0.8s ease';

      quillRef.current.style.transform = `translate(${x}px, ${y}px) scale(1.1) rotate(15deg)`;

    } else if (quillRef.current) {

      quillRef.current.style.animation = 'quillFlyAround 20s linear infinite';

      quillRef.current.style.transition = 'none';

      quillRef.current.style.transform = "";

    }

  }, [focusedInputPos]);
```

```

const handleFocus = (e) => {

  const rect = e.target.getBoundingClientRect();

  const x = rect.left + rect.width - 40;

  const y = rect.top + window.scrollY - 20;

  setFocusedInputPos({ x, y }); };

const handleBlur = () => {

  setFocusedInputPos(null); };

return (

  <div

    className="relative min-h-screen bg-cover bg-center py-20 px-6 font-serif"

    style={{ backgroundImage: "url('/assets/parchment.jpg')" }} >

    {/* Fireflies */}

    <div className="absolute inset-0 z-0 pointer-events-none">

      {[...Array(25)].map((_, i) => (

        <div

          key={i}

          className="absolute w-2 h-2 bg-yellow-300 rounded-full blur-md opacity-70
animate-float"

          style={{

            top: `${Math.random() * 100}%`,

            left: `${Math.random() * 100}%`,

            animationDuration: `${4 + Math.random() * 6}s`,

            animationDelay: `${Math.random() * 5}s`, }} /> ))}

        </div>

        {/* Flying Quill */}

{/* Glowing Contact Box */}

<div className="relative max-w-3xl mx-auto z-10">

  <div className="relative bg-yellow-100/10 backdrop-blur-md border border-yellow-
700 text-yellow-50 rounded-2xl shadow-2xl p-8 overflow-hidden contact-glow-box">

    <h2 className="text-3xl md:text-4xl font-bold text-yellow-200 mb-6 tracking-wider
border-b pb-2 border-yellow-600 font-cinzel">

      Contact Us

    </h2>

    <form className="space-y-6">

      <input

        type="text"

        placeholder="Your Name"

        className="w-full p-3 border border-yellow-700 rounded-md bg-yellow-100/20
text-yellow-100 placeholder-yellow-200 focus:outline-none focus:ring-2 focus:ring-yellow-
600 backdrop-blur-sm"

        onFocus={handleFocus}

        onBlur={handleBlur} />

      <input

        type="email"

        placeholder="Your Email"

        className="w-full p-3 border border-yellow-700 rounded-md bg-yellow-100/20
text-yellow-100 placeholder-yellow-200 focus:outline-none focus:ring-2 focus:ring-yellow-
600 backdrop-blur-sm"

        onFocus={handleFocus}
```

```
        onBlur={handleBlur}/>

<textarea

    placeholder="Your Message"

    rows="4"

    className="w-full p-3 border border-yellow-700 rounded-md bg-yellow-100/20
text-yellow-100 placeholder-yellow-200 focus:outline-none focus:ring-2 focus:ring-yellow-
600 backdrop-blur-sm"

    onFocus={handleFocus}

    onBlur={handleBlur}/>

    <button type="submit" className="bg-yellow-700 text-white px-6 py-2 rounded-md
font-semibold hover:bg-yellow-800 transition duration-300" > Send

    </button> </form> </div> </div>

const express = require('express');

const mongoose = require('mongoose');

const cors = require('cors');

const dotenv = require('dotenv');

// Load environment variables

dotenv.config();

// Initialize Express

const app = express();

// Middleware

app.use(cors({

    origin: 'http://localhost:3000',

    credentials: true

})); app.use(express.json());

// Routes

const contactRoutes = require('./routes/contactRoutes');
```

```
const userRoutes = require('./routes/userRoutes'); // Added this line

app.use('/api/contact', contactRoutes);

app.use('/api/users', userRoutes); // Added this line

// Health check

app.get('/', (req, res) => {

  res.send('Backend is running');

}); // Database connection

mongoose.connect(process.env.MONGO_URI, {

  useNewUrlParser: true,

  useUnifiedTopology: true

}).then(() => {

  console.log('\n 🟢 MongoDB Connected');

  // Start server

  const PORT = process.env.PORT || 5000;

  app.listen(PORT, () => {

    console.log(` 🚀 Server running on http://localhost:${PORT}`);

    console.log(` 📄 Contact form endpoint: POST  
http://localhost:${PORT}/api/contact/submit`);

    console.log(` 👤 User routes available under: http://localhost:${PORT}/api/users`); //
    Added this log  });})

.catch(err => {

  console.error('\n ❌ MongoDB Connection Error:', err.message);

  process.exit(1); });
```

3. User Interface Screenshots

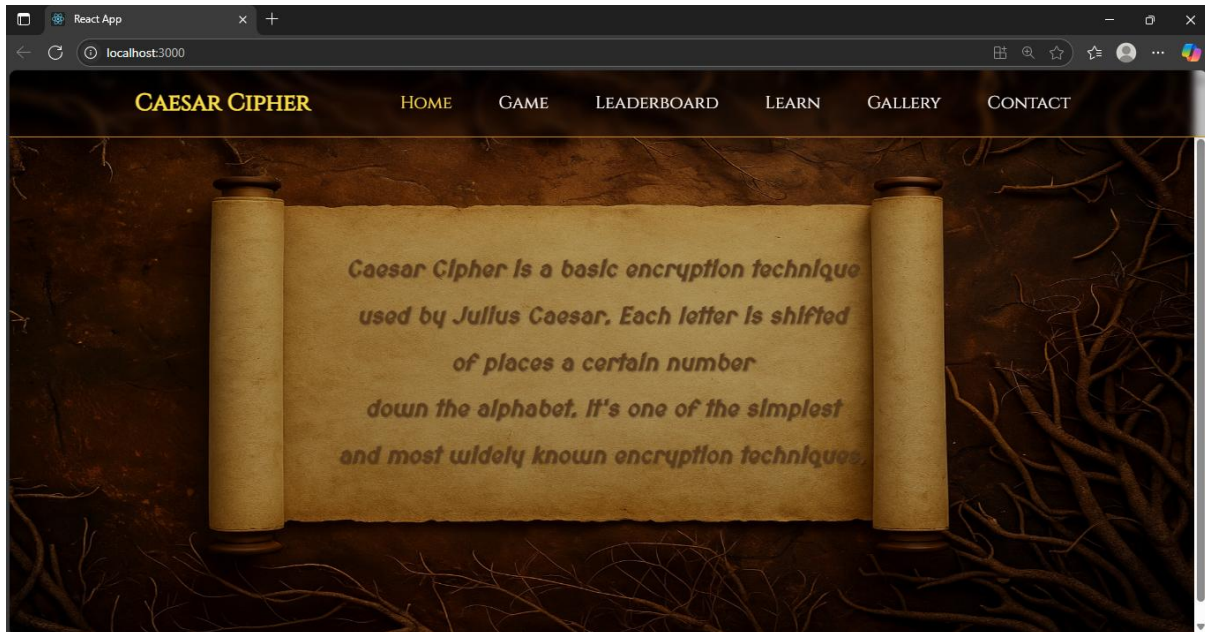


Figure 1 Unlock the Past! Explore the history of ciphers and encryption.

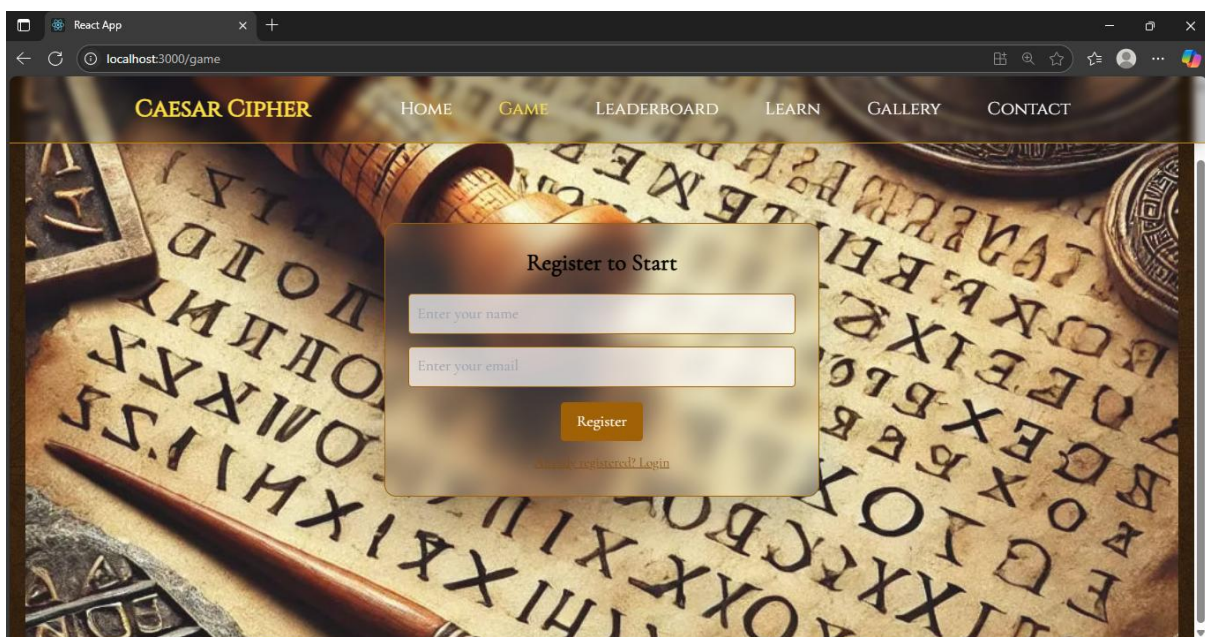


Figure 2 Delve into this endearing challenge

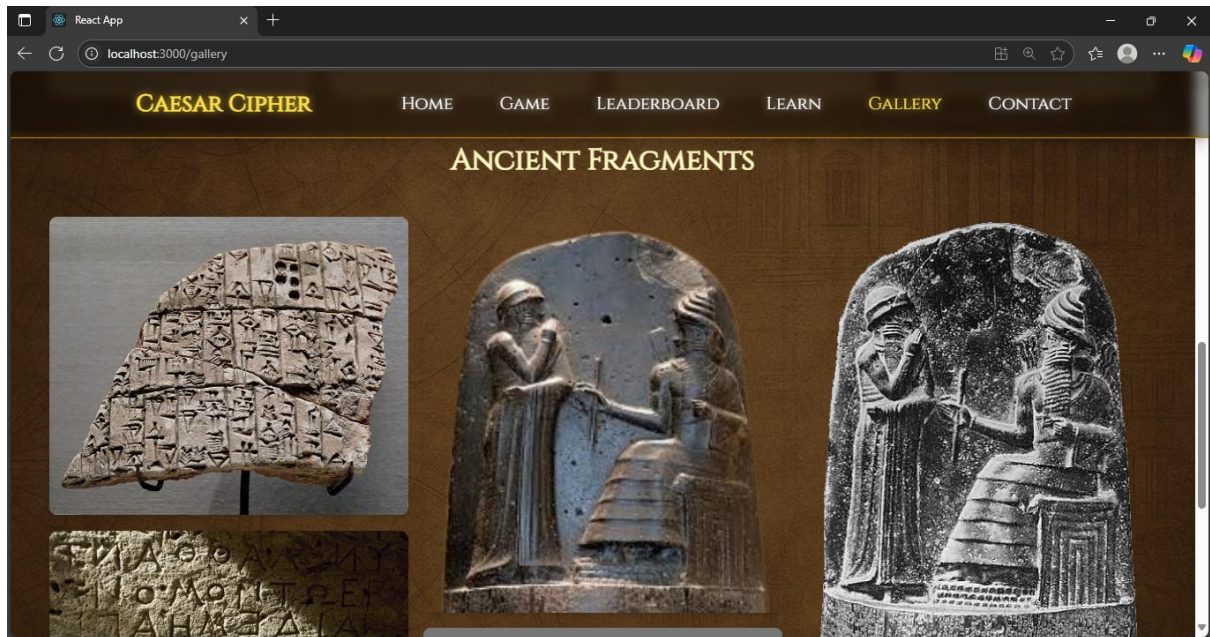


Figure 2 galleria of historians

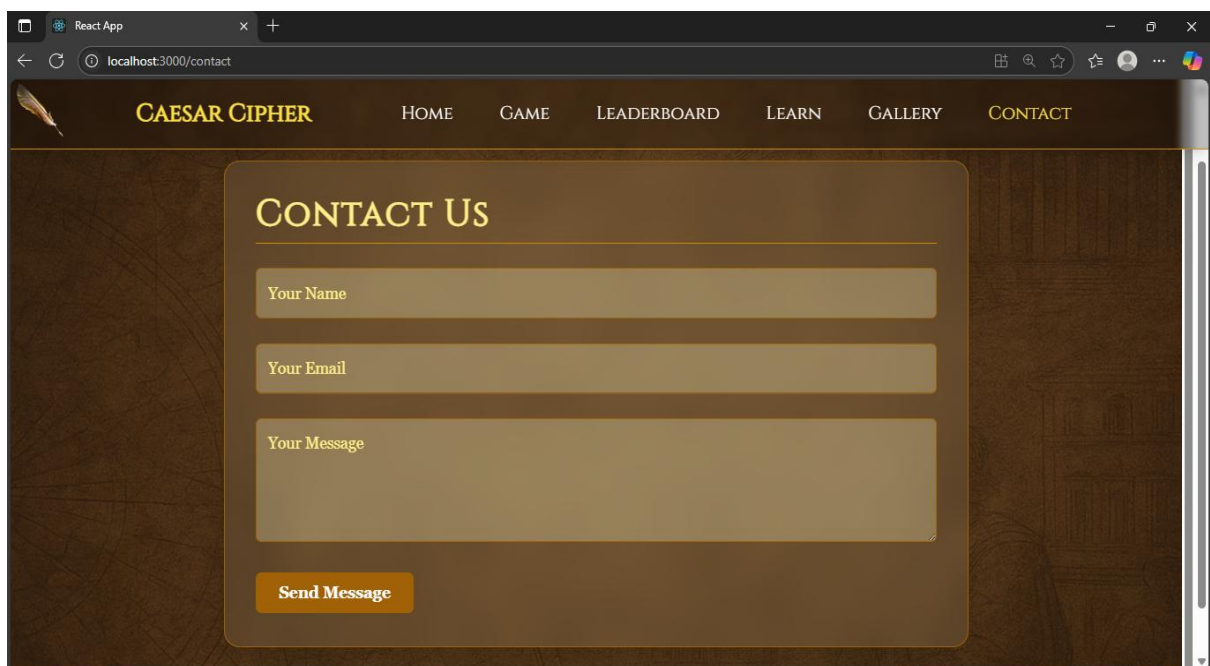


Figure 1 information is at your finger tip

4. Conclusion and Future Enhancement

Cryptic Chronicles: History of Code and Ciphers successfully merges education and interactivity to deliver an engaging learning experience centered on the evolution of cryptography. By combining historical insights with a hands-on cipher mini-game, the website makes complex encryption concepts approachable for users across varying levels of technical expertise. The platform allows users to explore the fascinating world of codes while enjoying the challenge of decrypting and encrypting text through intuitive game mechanics.

The project was developed as a full-stack web application using Node.js, Express, MongoDB, and Tailwind CSS. These technologies provide a responsive and visually appealing interface alongside a dynamic backend capable of handling user interactions. Additional features such as a gallery section showcase relevant visual content and add further depth to the learning experience. The site is designed with a mobile-first approach and features clear navigation, smooth performance, and user-friendly design principles to maximize accessibility across devices.

Looking ahead, several enhancements could elevate the platform further. Adding more cipher techniques to the mini-game and introducing difficulty levels would make the experience more challenging and rewarding. Incorporating hints or guided tutorials could assist users in solving complex ciphers, making the platform even more inclusive. Expanding with multimedia elements like videos and interactive infographics would enrich the educational value and boost engagement. A user authentication system could also be implemented to allow personalized experiences, progress tracking, and tailored challenges, transforming *Cryptic Chronicles* into a comprehensive learning tool for cryptography enthusiasts.